



DC – AI – Routing – Switching – Security – Wireless – Automation

SP-style Config in Apstra Freeform for VLAN Overlap Use-Cases

An HPE Juniper Networking Techpost

Elisabeth Rodrigues

2024-03-25

HPE



Contents

1	SP-style Config in Apstra Freeform for VLAN Overlap Use-Cases	3
1.1	Introduction	3
1.1.1	Refresher on EVPN VxLAN	3
1.1.2	What does Apstra Datacenter Reference Design Support?	4
1.2	Challenges of a Multi-Tenant Datacenter and How We Solve Them?	4
1.2.1	L3 Overlap	4
1.2.2	L2 Overlap	5
1.3	Example Description	5
1.4	Network Design With the Topology Editor	5
1.5	Resource Management	6
1.5.1	ASN Allocation	7
1.5.2	Loopback Allocation	8
1.5.3	IP Links Allocation	9
1.5.4	LACP Key Allocation	10
1.5.5	ASN Configuration template	11
1.5.6	Loopback Configuration template	12
1.5.7	eBGP Underlay Configuration Template	12
1.5.8	eBGP Overlay Configuration Template	13
1.5.9	ESI-LAG Configuration Template	14
1.6	Tenants design and configuration	15
1.6.1	Tenant Configuration template	16
1.6.2	Global Configuration Template	18
1.7	Conclusion	19
1.8	Glossary	19
1.9	Acknowledgements	19

List of Figures

1	lab topology	5
2	topology editor	6
3	Identify the spines with a tag	6
4	Create the following resources in the resource menu and allocate them in the blueprint. . .	7
5	Allocate an ASN to each managed device	7
6	Result	8
7	Loopback Allocation 1	8
8	Loopback Allocation 2	9
9	IP Links Allocation 1	9
10	IP Links Allocation 2	10
11	LACP Key Allocation 1	10
12	LACP Key Allocation 2	11
13	LACP Key Allocation 3	11
14	we define a Tenant in the Resource Management > Blueprint Resources	16

1 SP-style Config in Apstra Freeform for VLAN Overlap Use-Cases

Elisabeth Rodrigues - 03/25/2024

In high multi-tenant environments such as Service Providers, Hosting Providers, or just large enterprises, having to deal with multiple internal customers, efficient utilization of infrastructure is top of mind for network operations teams. While ensuring isolation and security among different users or departments, you also want to leverage network virtualization techniques to support full overlap of resources between those users. This not only speeds up the onboarding process for new customers, and companies you acquire, but it also preserves your infrastructure investment allowing you to deliver more with the same network infrastructure.

1.1 Introduction

This blog post describes the steps of implementing multi-tenancy in an EVPN-VxLAN data center fabric. We will delve into the intricacies of control-plane and data-plane details and how they are combined to fulfill layer 2 and layer 3 overlap. We will then use Apstra Freeform to design and deploy such a network.

The term “Multi-tenancy” applying to a broad range of domains, this blog post focuses on the resources overlap. What is outside of the scope of this post is the Multi-tenancy concept related to the user management part and any permission enforcement, in other words, the management plane. For that, watch out for an upcoming blog post for new capabilities coming soon on Juniper Apstra.

1.1.1 Refresher on EVPN VxLAN

EVPN supports multiple service interface types. Depending on the service interface defined, either all traffic on an interface, or traffic on a specific VLAN, or traffic on a list/range of VLANs can be mapped to a Bridge Domain (BD). That Bridge Domain is then associated with an EVPN instance (EVI) for forwarding VxLAN-encapsulated traffic across the IP fabric. An EVPN instance represents an L2 VPN instance on a switch, also known as a MAC-VRF in Junos implementation.

How does the segregation happen at the Control-plane vs. Data-plane level?

- Control-plane separation: depending on the EVPN service interface defined, EVIs are assigned import/-export Route Targets (RTs) and Route Distinguisher (RD).
- Data-plane separation: VNI field of the VxLAN header is used during packet lookup to place onto the right Bridge-Domain/VLAN.

The L3 EVPN instance serves the same role as an IP VPN (VRF), but in this case is an EVPN VRF, sometimes referred to as Type-5 VRF due to the EVPN Type-5 routes. This ensures L3 overlap. What we would like to do is to push this further by also leveraging EVPN for L2 overlap.

A MAC-VRF or EVI can offer 3 types of service interface types:

- VLAN-based: Single VLAN broadcast domain and single bridge domain. Every VLAN-id is mapped to one Bridge Domain, mapped to one VNI, and one has its own route-target.
- VLAN-aware: Multiple VLAN broadcast domains with each VLAN having its own bridge domain. Every VLAN is mapped to one VNI and can have its own route-target. This is the preferred option for most enterprises.

- **VLAN-bundle:** Multiple VLAN broadcast domains sharing a single bridge domain, MAC addresses must be unique. All VLAN-ids are mapped to the same VNI and the same route-target. The VLAN-id is encapsulated in the VXLAN packet. This option helps to overlap the VLAN-ids but also to deliver a higher VLAN scale. This is a typical option for hosting companies.

1.1.2 What does Apstra Datacenter Reference Design Support?

On Juniper fabrics, Apstra Datacenter reference design configures EVPN VXLAN with multiple EVPN VRFs and a single MAC-VRF of service interface type VLAN-aware. The Junos configuration style used is “Enterprise Style”.

Enterprise Style vs SP Style with Junos ?

Enterprise style is simpler and needs less configuration.

Here is an example of an interface with multiple VLANs using enterprise-style :

```
1 interfaces ael unit 0 {
2   family ethernet-switching {
3     interface-mode trunk;
4     vlan {
5       members [ vn100 vn200 ];
6     }
7   }
```

SP Style Offers greater flexibility but requires more configuration steps.

Here is the same example using SP-Style :

```
1 interfaces ael unit 0 {
2   flexible-vlan-tagging;
3   encapsulation flexible-ethernet-services;
4   unit 100 {
5     vlan-id 100;
6   }
7   unit 200 {
8     vlan-id 200;
9   }
```

On top of that, each logical interface must be added to the appropriate bridge-domain.

1.2 Challenges of a Multi-Tenant Datacenter and How We Solve Them?

When you run a highly multi-tenant environment such as a shared EVPN-VxLAN fabric hosting several customers with the goal to offer each of them as much flexibility as possible, you often have to deal with L2 and L3 overlap. Let's dissect each one of them.

1.2.1 L3 Overlap

This happens when the tenants use the same IP space.

Solution: this can be easily solved using EVPN T5 VRFs. Each tenant will have one or multiple VRFs ensuring IP subnet uniqueness and avoiding any duplicate.

1.2.2 L2 Overlap

This happens when the tenants use the same VLAN-id space.

We can distinguish 2 use cases:

- There is no need for VLAN-id overlap at the leaf level. This can be easily addressed with the Apstra Datacenter reference design. In that case, each virtual network is assigned a different VNI and the VLAN-id association is configured for each leaf.
- There is a need for VLAN-id overlap at the leaf level. There are several ways to address it:

We can use Configlets in the Datacenter Reference design to push additional configurations based on the Junos functionality called “VLAN Rewrite”. This additional configuration allows you to define translation rules where a given VLAN ID is rewritten to use another VLAN ID. It is a simple configlet, and represents a good solution for a temporary use, typically during a migration time. We can use SP-Style configuration. - Level 3 multi-tenancy : each tenant is separated by a L3 domain.

1.3 Example Description

This example utilizes Apstra release 4.2.1

Apstra is the Single Source of Truth.

In this example, ESXi-110 and ESXi-114 use the same vlan-id but belong to different tenants and must be encapsulated in different VNIs.

Here is the lab topology:

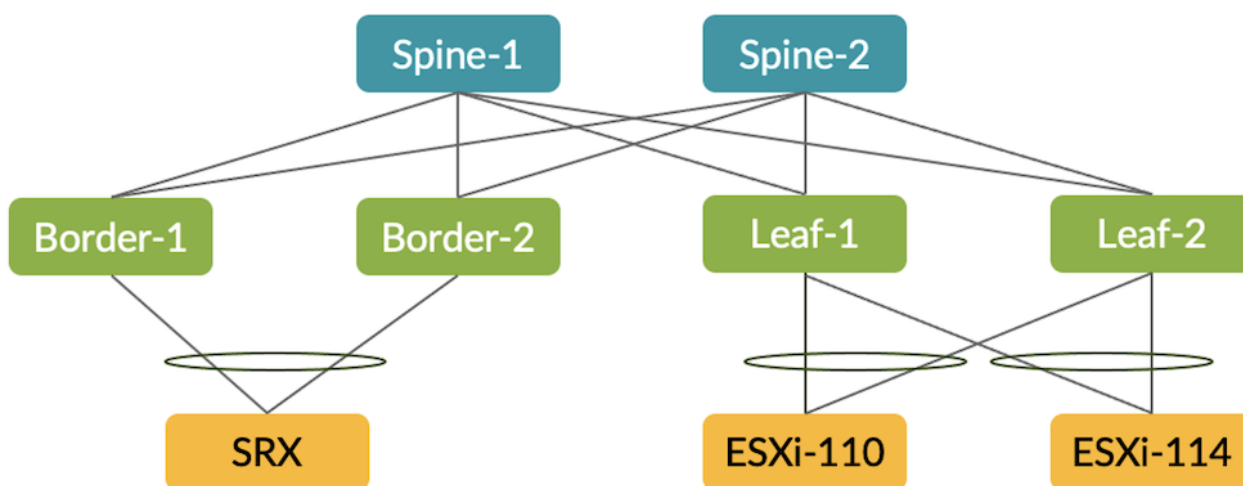


Figure 1: lab topology

1.4 Network Design With the Topology Editor

Create a new Freeform blueprint and design the topology with the topology editor:

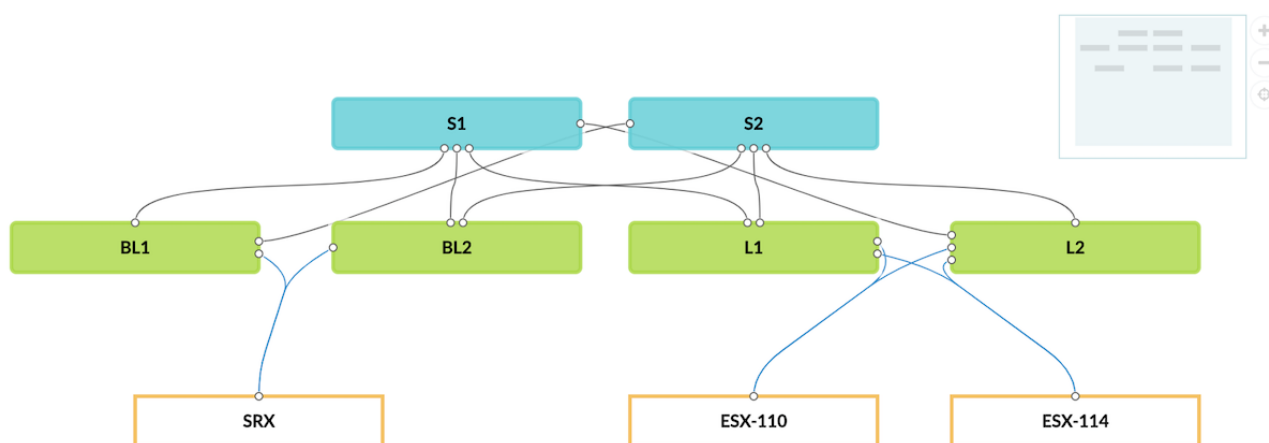


Figure 2: topology editor

Identify the spines with a tag:

☐	Name	Type	Tags	Deploy Mode	Device Profile	S/N	Hostname
0 selected							
☐	BL1	INTERNAL		Deploy	Juniper QFX10002-36Q	Not assigned	BL1
☐	BL2	INTERNAL		Deploy	Juniper QFX10002-36Q	Not assigned	BL2
☐	ESX-110	EXTERNAL		Deploy	N/A	N/A	ESX-110
☐	ESX-114	EXTERNAL		Deploy	N/A	N/A	ESX-114
☐	L1	INTERNAL		Deploy	Juniper QFX5120-48Y	Not assigned	L1
☐	L2	INTERNAL		Deploy	Juniper QFX5120-48Y	Not assigned	L2
☐	S1	INTERNAL	Spine	Deploy	Juniper QFX5200-32C	Not assigned	S1
☐	S2	INTERNAL	Spine	Deploy	Juniper QFX5200-32C	Not assigned	S2
☐	SRX	EXTERNAL		Deploy	N/A	N/A	SRX

Figure 3: Identify the spines with a tag

1.5 Resource Management

Create the following resources in the resource menu and allocate them in the blueprint.

Name ▾	Type ▾
ASN	ASN
Lo0	IPv4
Links	IPv4
RT-RD	Integer
LACP-KEY	Integer

Figure 4: Create the following resources in the resource menu and allocate them in the blueprint.

In the Resource Management Menu, Blueprint Resources, create a group named Cloud.

1.5.1 ASN Allocation

Allocate an ASN to each managed device, all spines, all leaves, and border leaves.

Create Resource Generator

Name *

ASN

Container Type

Group

Container

Cloud

Scope *

1 | node('system', system_type= 'internal',name='target')

Type

☐ IPv4 ☐ Host IPv4 ☐ IPv6 ☐ Host IPv6 ☒ ASN ☐ VNI ☐ VLAN ☐ Integer

Allocation Group

ASN

Create

Figure 5: Allocate an ASN to each managed device

The result is the following.

Name	Type	Value	Generated By	Allocated From	Assigned To
ASN	ASN	65005	ASN	ASN	L2
ASN	ASN	65002	ASN	ASN	S1
ASN	ASN	65000	ASN	ASN	BL1
ASN	ASN	65004	ASN	ASN	L1
ASN	ASN	65001	ASN	ASN	BL2
ASN	ASN	65003	ASN	ASN	S2

Figure 6: Result

1.5.2 Loopback Allocation

Allocate a loopback IP to each managed device, all spines, all leaves, and border leaves.

Edit Resource Generator

Name *

Lo0

Container Type

Group

Scope *

1 ▾ node('system', system_type="internal", name='target')

Type

IPv4

Allocation Group

Lo0

Subnet Prefix Length

32

Figure 7: Loopback Allocation 1

The result is the following.

Name	Type	Value	Generated By	Allocated From	Assigned To
Lo0	IPv4	10.0.0.5/32	Lo0	Lo0	S1
Lo0	IPv4	10.0.0.3/32	Lo0	Lo0	BL1
Lo0	IPv4	10.0.0.2/32	Lo0	Lo0	L2
Lo0	IPv4	10.0.0.1/32	Lo0	Lo0	L1
Lo0	IPv4	10.0.0.4/32	Lo0	Lo0	BL2
Lo0	IPv4	10.0.0.0/32	Lo0	Lo0	S2

Figure 8: Loopback Allocation 2

1.5.3 IP Links Allocation

Allocate a /31 subnet to each internal link between all spines and all leaves / border leaves.

Create Resource Generator

Name *

Links

Container Type

Group

Container

Cloud

Scope *

1 node('link', role='internal', name='target')

Type

☒ IPv4
 ☐ Host IPv4
 ☐ IPv6
 ☐ Host IPv6
 ☐ ASN
 ☐ VNI
 ☐ VLAN
 ☐ Integer

Allocation Group

Links

Subnet Prefix Length

31

Create

Figure 9: IP Links Allocation 1

The result is the following.

Name	Type	Value	Generated By	Allocated From	Assigned To
Links	IPv4	192.168.0.0/31	Links	Links	L2<->S2
Links	IPv4	192.168.0.10/31	Links	Links	L2<->S1
Links	IPv4	192.168.0.8/31	Links	Links	L1<->S1
Links	IPv4	192.168.0.12/31	Links	Links	BL1<->S2
Links	IPv4	192.168.0.14/31	Links	Links	L1<->S2
Links	IPv4	192.168.0.2/31	Links	Links	BL1<->S1
Links	IPv4	192.168.0.6/31	Links	Links	S1<->BL2
Links	IPv4	192.168.0.4/31	Links	Links	BL2<->S2

Figure 10: IP Links Allocation 2

1.5.4 LACP Key Allocation

Allocate an integer to each external device. This value will be used to generate a unique LACP key to each connected device.

Create Resource Generator

Name *

LACP-Key

Container Type

Group

Container

Cloud

Scope *

1 ▾ node('system', system_type='external', name='target')

Type

☐ IPv4
☐ Host IPv4
☐ IPv6
☐ Host IPv6
☐ ASN
☐ VNI
☐ VLAN
☒ Integer

Allocation Group

LACP-KEY

Create

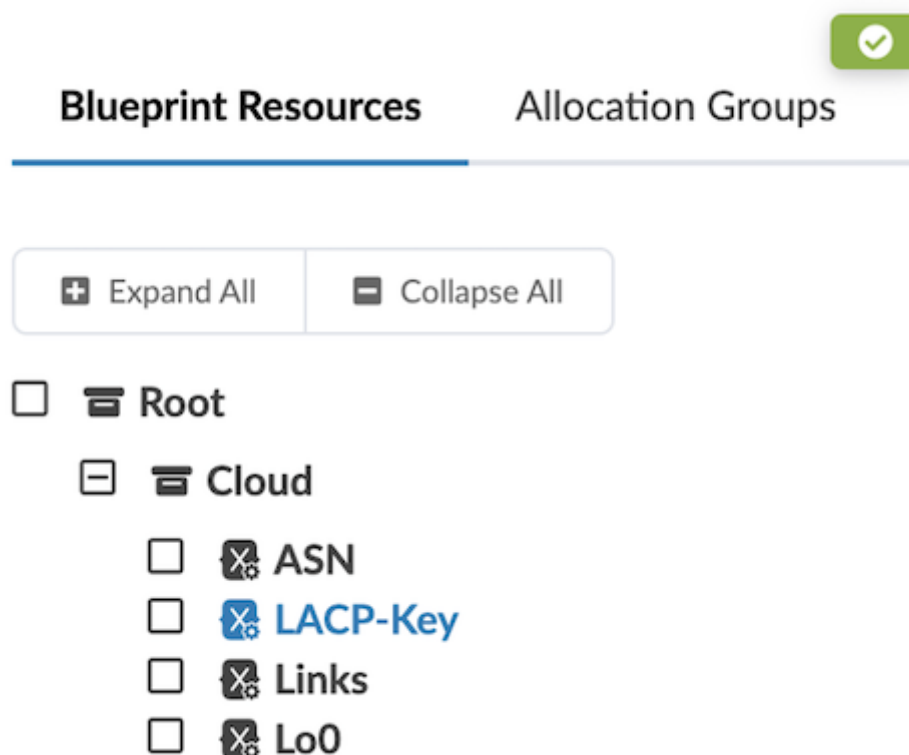
Figure 11: LACP Key Allocation 1

The result is the following:

Name	Type	Value	Generated By	Allocated From	Assigned To
LACP-Key	Integer	2	LACP-Key	LACP-KEY	ESX-114
LACP-Key	Integer	3	LACP-Key	LACP-KEY	SRX
LACP-Key	Integer	1	LACP-Key	LACP-KEY	ESX-110

Figure 12: LACP Key Allocation 2

All the resources created appear in the Blueprint resources:

**Figure 13:** LACP Key Allocation 3

Config Templates

After allocating the resources, we need to apply the Junos command lines to configure the devices.

Create the following configuration templates.

1.5.5 ASN Configuration template

Create the following Configuration Template “Cloud_ASN.jinja”

```

1 {% set asn = function.get_resource_value(resources, 'ASN', 'Cloud') %}
2
3 routing-options {
4     replace: autonomous-system {{asn}};
5 }

```

Rendered configuration on L1

```
1  routing-options {
2      replace: autonomous-system 65102;
3  }
```

1.5.6 Loopback Configuration template

Create the following Config Template “Cloud-Loopback.jinja”

```
1  {% set loopback_ipv4 = function.get_resource_value(resources, 'Lo0', 'Cloud') %}
2  interfaces {
3      lo0 {
4          unit 0 {
5              family inet {
6                  address {{loopback_ipv4}};
7              }
8          }
9      }
10 }
11 {% set router_id = loopback_ipv4|to_ip %}
12 routing-options {
13     replace: router-id {{router_id}};
14 }
```

Rendered configuration on L1

```
1  interfaces {
2      lo0 {
3          unit 0 {
4              family inet {
5                  address 10.0.0.4/32;
6              }
7          }
8      }
9  }
10 routing-options {
11     replace: router-id 10.0.0.4;
12 }
```

1.5.7 eBGP Underlay Configuration Template

Create the following Config Template “Cloud_eBGP-Underlay.jinja”

```
1  {% set ASN = function.get_resource_value(resources, 'ASN', 'Cloud') %}
2  protocols {
3      bgp {
4          group underlay {
5              type external;
6              local-as {{ASN}};
7              export Export-L0;
8              multipath {
9                  multiple-as;
10             }
11             {% for interface_name, iface in interfaces.items() %} #for each interface
12                 {% if iface.get('neighbor_interfaces') %} #if the interface is connected to a
13                     neighbour
14                     {% for nb in iface.get('neighbor_interfaces') %} #get neighbour information
15                         {% if (nb['link_role'] == "internal") %} #if the neighbour is an internal
16                             system, get its ASN and interface IP
17                             {% set PEER_ASN = function.get_resource_value(all_resources[nb['system_id']], 'ASN', 'Cloud') %}
18                             neighbor {{nb['ipv4_address']}} {
19                                 peer-as {{PEER_ASN}};
20                             }
21                         {% endif %}
22                     {%- endfor -%}
23                 {%- endif -%}
24             {% endfor %}
25         }
26     }
27 }
```

```

25 }
26
27 policy-options {
28     policy-statement Export-LO {
29         from interface lo0.0;
30         then accept;
31     }
32 }

```

Rendered configuration on L1

```

1  protocols {
2      bgp {
3          group underlay {
4              type external;
5              local-as 65102;
6              export Export-LO;
7              multipath {
8                  multiple-as;
9              }
10             neighbor 192.168.0.15 {
11                 peer-as 65101;
12             }
13             neighbor 192.168.0.11 {
14                 peer-as 65103;
15             }
16         }
17     }
18 }
19 policy-options {
20     policy-statement Export-LO {
21         from interface lo0.0;
22         then accept;
23     }
24 }

```

1.5.8 eBGP Overlay Configuration Template

Create the following Config Template “Cloud_eBGP-Overlay.jinja”

```

1  {% set ASN = function.get_resource_value(resources, 'ASN', 'Cloud') %}
2  {% set Lo0 = function.get_resource_value(resources, 'Lo0', 'Cloud') %}
3  {% set Lo0_nomask = Lo0|to_ip %}
4  protocols {
5      bgp {
6          group overlay {
7              type external;
8              local-as {{ASN}};
9              local-address {{Lo0_nomask}};
10             multipath {
11                 multiple-as;
12             }
13             multihop {
14 {% if 'Spine' in system_tags %}
15                 no-nexthop-change; #Specific configuration on Spine
16 {% endif %}
17                 ttl 1;
18             }
19             family evpn {
20                 signaling {
21                     loops 2;
22                 }
23             }
24             vpn-apply-export;
25             {% for interface_name, iface in interfaces.items() %} #for each interface
26                 {% if iface.get('neighbor_interfaces') %} #if the interface is connected to a
27                     neighbour
28                     {% for nb in iface.get('neighbor_interfaces') %} #get neighbour information
29                         {% if (nb['link_role'] == "internal") %} #if neighbour is an internal system,
30                             get loopback IP and ASN
31                             {% set PEER_ASN = function.get_resource_value(all_resources[nb['system_id']], '
32                                 ASN', 'Cloud') %}
33                             {% set PEER_LO = function.get_resource_value(all_resources[nb['system_id']], 'Lo0
34                                 ', 'Cloud') %}
35                             {%set PEER_LO_nomask = PEER_LO|to_ip %}

```

```

32     neighbor {{PEER_LO_nomask}} {
33         peer-as {{PEER_ASN}};
34     }
35     {% endif %}
36     {%~ endfor ~%}
37     {%~ endif ~%}
38     {% endfor %}
39 }
40 }
41 }

```

Rendered configuration on L1

```

1  protocols {
2      bgp {
3          group overlay {
4              type external;
5              local-as 65102;
6              local-address 10.0.0.4;
7              multipath {
8                  multiple-as;
9              }
10             multihop {
11                 ttl 1;
12             }
13             family evpn {
14                 signaling {
15                     loops 2;
16                 }
17             }
18             vpn-apply-export;
19             neighbor 10.0.0.5 {
20                 peer-as 65101;
21             }
22             neighbor 10.0.0.1 {
23                 peer-as 65103;
24             }
25         }
26     }
27 }

```

1.5.9 ESI-LAG Configuration Template

Create the following Config Template “Cloud_LAG.jinja”

```

1  {% for interface_name, iface in interfaces.items() %} #for each interface
2      {% if iface.composed_of %} #if the interface is member of a LAG
3          {% for nb in iface.get('neighbor_interfaces') %} #get neighbour LACP_key
4          {% set LACP_KEY = function.get_resource_value(all_resources[nb['system_id']], 'LACP-Key', 'Cloud
5          interfaces {{interface_name}}{
6              esi {
7                  auto-derive {
8                      lacp-pe-system-id-and-admin-key;
9                  }
10                 all-active;
11             }
12             aggregated-ether-options {
13                 lacp {
14                     system-id 00:00:00:00:00:10;
15                     admin-key {{LACP_KEY}};
16                 }
17             }
18             {% endfor %} }
19         {% endif %}
20     {% endfor %}

```

Rendered configuration on L1:

```

1  interfaces ael{
2      esi {
3          auto-derive {
4              lacp-pe-system-id-and-admin-key;
5          }

```

```
6         all-active;
7     }
8     aggregated-ether-options {
9         lacp {
10             system-id 00:00:00:00:00:10;
11             admin-key 2;
12         }
13     }
14 }
15 interfaces ae2{
16     esi {
17         auto-derive {
18             lacp-pe-system-id-and-admin-key;
19         }
20         all-active;
21     }
22     aggregated-ether-options {
23         lacp {
24             system-id 00:00:00:00:00:10;
25             admin-key 3;
26         }
27     }
28 }
```

1.6 Tenants design and configuration

Each tenant is a unique MAC-VRF.

Each MAC-VRF will have the following attributes:

- A VRF Target
- A Route-Distinguisher for each leaf/border leaf
- A list for VLANs
- Optionally, an IRB for each defined VLAN

Here is how we define a Tenant in the Resource Management > Blueprint Resources:

The screenshot displays the Apstra Resource Management interface. On the left, a sidebar shows a tree view of the configuration hierarchy: Root > Cloud > Tenants. Under 'Tenants', 'Tenant-A' is selected and highlighted with a red box. The main panel shows the configuration for 'Tenant-A'. The 'Details' section shows 'Data' as an empty object and 'Assignments' as 'Not Assigned'. The 'Resources' section shows a table of resources. The table has columns: Name, Type, Value, Generated By, Allocated From, Assigned To, and Actions. The resources listed are: RT-RD (Integer, 1), VLAN-20 (VLAN, 20), VLAN-20 (Host IPv4, 10.0.20.254/24), VLAN-100 (VLAN, 100), and VLAN-100 (Host IPv4, 10.0.100.254/24). The 'Assigned To' column shows that 'ESX-114' is assigned to 'VLAN-20' and 'VLAN-100'. The 'Actions' column contains icons for edit, delete, and refresh. Red boxes highlight the 'Tenants' tree, the 'RT-RD' resource, the 'VLAN-20' resource, and the 'ESX-114' resource assigned to 'VLAN-20'.

Figure 14: we define a Tenant in the Resource Management > Blueprint Resources

The physical interfaces are assigned to a tenant and a list VLANs by using the assignment button on the right. Here we can see that ESX-114 is assigned to VLAN-20 and Tenant-A.

1.6.1 Tenant Configuration template

```

1  {% if 'Spine' not in system_tags %} #for all leaves
2  {%- set all_vrf = function.get_resource(resources,'Tenants','Cloud')%} #get the list of
   tenants
3  {%- if all_vrf|length -%}
4  {% set loopback_ipv4 = function.get_resource_value(resources, 'Lo0', 'Cloud' ) %}
5  {% set loopback_no_mask = loopback_ipv4|to_ip %}
6  {% for get_vrfs in all_vrf['groups'] -%}
7  {%- set ns = namespace() %}
8  {%- set ns.name = "" %}
9  {%- set ns.vlan_id = "" %}
10  {%- for vrf_name in get_vrfs['label'] -%}
11  {%- set ns.name = ns.name + vrf_name %}
12  {%- endfor -%}
13  {%- set RT_RD = function.get_resource_value(resources,'RT-RD','Cloud','Tenants', ns.
      name) %}
14
15  routing-instances VRF-{{ns.name}} { #configure the L3 VRF
16    instance-type vrf;
17    route-distinguisher {{loopback_no_mask}}:1000{{RT_RD}};
18    vrf-target target:65000:10000{{RT_RD}};
19    vtep-source-interface lo0.0;
20    routing-options {
21      multipath;
22      auto-export;
23    }
24    protocols {
25      evpn {
26        ip-prefix-routes {
27          advertise direct-nexthop;
28          encapsulation vxlan;
29          vni 10000{{RT_RD}};

```

```

30     }
31   }
32 }
33 }
34
35
36 routing-instances {{ns.name}} { #configure the L2 MAC-VRF
37   instance-type mac-vrf;
38   service-type vlan-aware;
39   route-distinguisher {{loopback_no_mask}}:{{RT_RD}};
40   vrf-target target:65000:{{RT_RD}};
41   vtep-source-interface lo0.0;
42   protocols {
43     evpn {
44       encapsulation vxlan;
45       duplicate-mac-detection {
46         auto-recovery-time 9;
47       }
48       extended-vni-list all;
49     }
50   }
51 }
52 {% set TENANTS_INFO = function.get_all_resources(resources,ns.name, 'Cloud', 'Tenants') %}
53 {% for INFOS in TENANTS_INFO %}
54   {%- for item in INFOS['resources'] -%}
55     {%- if item['resource_type'] == 'vlan' %} #check if a vlan id defined
56       {%- set ns.vlan_id = item['value'] %}
57       {%- set ns.ip = "" -%}
58       {%- set vlan_name = item['label'] %}
59       {%- for item in INFOS['resources'] -%}
60         {%- if item['resource_type'] == 'host_ip' and item['label'] == vlan_name
61           -%} #get the vlan irb (if exists)
62           {%- set ns.ip = item['value'] -%}
63         {% endif %}
64         {%- endfor %}
65 routing-instances {{ns.name}} { #configure vlan inside the MAC-VRF and, if it exist, the l3-
66   interface (irb)
67     vlans {
68       {{item['label']}} {
69         vlan-id none;
70         {%- if ns.ip != "" %}
71         l3-interface irb.{{RT_RD}}:{{ns.vlan_id}};
72         {%- endif %}
73         vxlan {
74           vni {{RT_RD}}:{{ns.vlan_id}};
75         }
76       }
77     }
78   {%- if ns.ip != "" %} #if irb exists, configure irb interface and assign it to L3
79   VRF
80   interfaces {
81     irb {
82       unit {{RT_RD}}:{{ns.vlan_id}} {
83         family inet address {{ns.ip}};
84         mac 00:ba:be:00:00:01;
85       }
86     }
87   }
88   routing-instances VRF-{{ns.name}} {
89     interface irb.{{RT_RD}}:{{ns.vlan_id}};
90   }
91   {%- endif %}
92   {% for members in item['assigned_to'] %} //check assigned interface
93     {%- for interface_name, iface in interfaces.items() -%}
94       {%- if iface.get('neighbor_interfaces') -%}
95         {%- for neighbor in iface.get('neighbor_interfaces') -%}
96           {%- if (neighbor['system_id'] == members) and (neighbor['link_role']
97             == "external") %}
98             {%- if not iface.part_of %}
99             interfaces {#configure unit for that interface
100               {{ interface_name }} {
101                 flexible-vlan-tagging;
102                 encapsulation flexible-ethernet-services;
103                 unit {{ns.vlan_id}} {
104                   encapsulation vlan-bridge;
105                   vlan-id {{ns.vlan_id}};
106                 }
107               }
108             }
109             routing-instances {{ns.name}} { // assigned interface to vlan

```

```

107     vlans {
108         {{item['label']}} {
109             interface {{ interface_name }}.{{ns.vlan_id}};
110         }
111     }
112 }
113
114             {% endif %}
115             {% if iface.composed_of %}
116 interfaces {#Freeform needs unit 0 to monitor interfaces, so we configure a dummy unit 0 here
117     {{ interface_name }} {
118         unit 0 {
119             vlan-id 4094;
120         }
121     }
122 }
123             {% endif %}
124             {%~ endif %}
125             {%~ endfor %}
126             {%~ endif %}
127             {%~ endfor %}
128             {%~ endfor %}
129             {%~ endif %}
130             {%~ endfor %}
131             {%~ endfor %}
132             {%~ endfor %}
133             {%~ endif %}
134 {% endif %}

```

Rendered Config on L1 for Tenant A:

```

1  routing-instances VRF-Tenant-A {
2      instance-type vrf;
3      route-distinguisher 10.0.0.4:10001;
4      vrf-target target:65000:100001;
5      vtep-source-interface lo0.0;
6      routing-options {
7          multipath;
8          auto-export;
9      }
10     protocols {
11         evpn {
12             ip-prefix-routes {
13                 advertise direct-nexthop;
14                 encapsulation vxlan;
15                 vni 100001;
16             }
17         }
18     }
19 }

```

1.6.2 Global Configuration Template

Change the junos configuration template “junos_configuration.jinja” as follow :

```

1  {# junos_system.jinja handles the system hostname #}
2  {% include "junos_system.jinja" %}
3
4  {# junos_chassis.jinja handles chassis options, such as fpc config for
5   channelized port break-outs on certain device platforms. This also handles
6   aggregate-devices ethernet device-count for port-channel (aeX) interfaces. #}
7  {% include "junos_chassis.jinja" %}
8
9  {# junos_interfaces.jinja handles front-panel interface configuration, including
10   interface description, ipv4/ipv6 address assignment, and physical link properties
11   derived from device profiles. #}
12  {% include "junos_interfaces.jinja" %}
13
14  {# junos_protocols.jinja initiates LLDP collection on all ports for telemetry
15   purposes #}
16  {% include "junos_protocols.jinja" %}
17
18  {#configure LAG with ESI#}
19  {% include "Cloud-LAG.jinja" %}
20

```

```
21 {#configure loopback#}  
22 {% include "Cloud-Loopback.jinja" %}  
23  
24 {#configure ASN#}  
25 {% include "Cloud_ASN.jinja" %}  
26  
27 {#Underlay eBGP#}  
28 {% include "Cloud_eBGP-Underlay.jinja" %}  
29  
30 {#Overrlay eBGP#}  
31 {% include "Cloud_eBGP-Overlay.jinja" %}  
32  
33 {#MAC-VRFs#}  
34 {% include "Cloud-MAC-VRF.jinja" %}
```

1.7 Conclusion

Apstra Freeform allows you to design and manage multi-tenant data center networks with an unprecedented level of design flexibility. The unique Apstra Freeform capability of resource generators is a powerful capability that you can use to streamline the allocation of network resources. This functionality facilitates the creation of highly customized and efficient network configurations by dynamically generating necessary parameters such as Autonomous System Numbers (ASNs), loopback addresses, and VLAN IDs across the entire fabric. By doing so, Apstra Freeform significantly reduces the manual overhead typically associated with configuring and managing complex, multi-tenant networks

1.8 Glossary

- ASN: Autonomous System Number
- BGP: Border Gateway Protocol
- EVPN: Ethernet Virtual Private Network
- ESI: Ethernet Segment Identifier
- EVI: EVPN Instance
- NOS: Network Operating System
- RBAC: Role Based Access Control
- RD: Route Distinguisher
- RT: Route Targer
- VXLAN:
- ZTP: Zero Touch Provisionning

1.9 Acknowledgements

I would like to thank Mehdi Abdelouahab for reviewing and adding valuable and insightful content to this article.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com

HPE

